

Assignment - 02
 Software Engineering & System Design
 Name - Reetika Pratap Singh
 ERPID - 0241BTC047

Q1 System design vs Software Design + Scaling?

→ Software design :-

Software design focuses on the internal structure of software, including classes, functions, modules, and design patterns.

It focuses on :-

→ internal structure of a software design

→ Algorithms, data structure, class design

System design :-

System design is the process of defining the architecture, components, modules, interfaces, and data flow of a complete system.

It focuses on :-

→ High level structure

→ scalability, reliability, performance

→ Integration of multiple components

Difference

Parameters

System Design

Software Design

Scope

Level

High-level

Low-level

Focuses

Architecture

Code structure

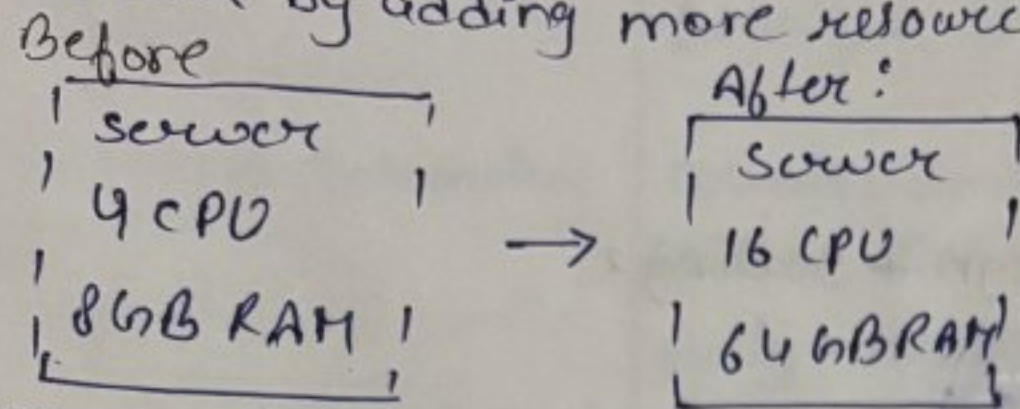
Example

Designing backend system

Designing algorithm

Vertical scaling

Vertical scaling is the process of increasing the capacity of a single server by adding more resources such as CPU, RAM or storage.

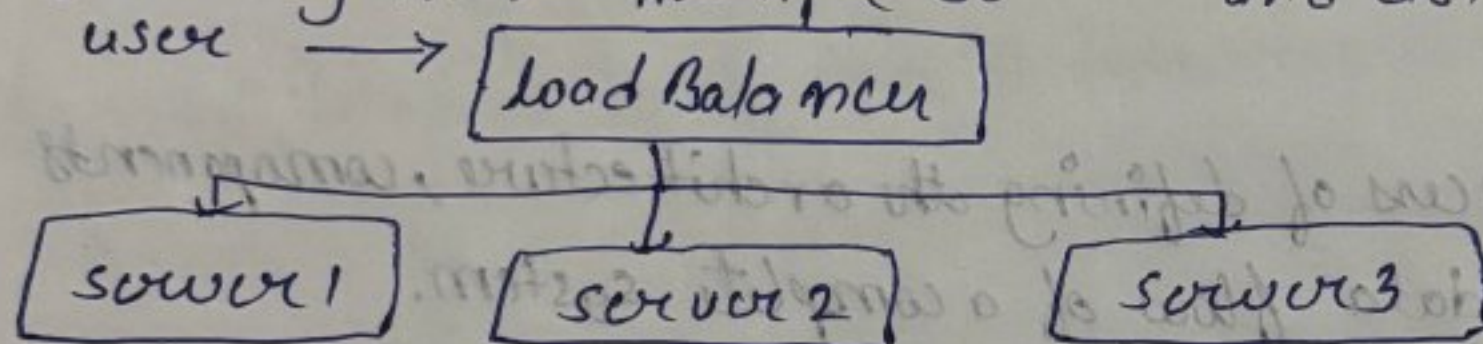


Pros: → simple, No distributed complexity

Cons: Expensive, limited capacity, single point of failure

Horizontal scaling:-

Horizontal scaling is the process of increasing system capacity by adding more multiple servers and distributing



Pros:- High fault tolerance, Infinite scalability

Cons:- Requires distributed coordination

Trade off summary factor

	Vertical	Horizontal
Cost	Expensive hardware	More machines
fault tolerance	low	high
Performance	limited	high

System Quality Attributes

(a) Availability →

The percentage of time a system is operational and accessible

formulae:

$$\text{Availability} = \text{Uptime} / (\text{Uptime} + \text{Downtime}) \times 100\%$$

"Nines" of Availability →

level

%

99%

Two nines

Downtime/Year

~ 3.65 days

99.9%

Three nines

~ 8.76 hours

99.99%

four nines

~ 52.6 minutes

99.999%

five nines

~ 5.26 minutes

Achieving high Availability via Redundancy.

Active - Passive Backup server takes over when primary fails.

Active - Active: Multiple servers handles traffic simultaneously

Replication: Data copied across nodes to avoid data loss.

Health checks + Auto-restart using tools like Kubernetes.

(b) Reliability

Reliability = Probability system works without failure over time

Metrics:

MTBF (Mean Time Between failures): average time between consecutive failures.

MTTR (Mean Time to Repair): Average time to restore a system after failure.

$$\text{MTTR} = \frac{\text{Total Downtime}}{\text{Number of failures}}$$

Definition	Reliability	Availability
	Consistent correct behavior	system is up and accessible
focus	Correctness & consistency	uptime
formula-based on	MTBF	$\text{uptime} / (\text{uptime} + \text{downtime})$
Ex	System never wrong data	system is accessible 99.99% of time

A system can be available but unreliable (up but giving wrong results)

1) Latency

The time taken from sending a request to receiving a response.

Percentile Metrics:

P50: 50% of requests are served within this time (median experience).

P95: 95% of requests are served within time (most user experience)

P99: 99% of requests are served within this time (worst-case / outliers).

P99 is most important for SLA guarantees

How CDN and Caching Reduce latency

CDN: stores static assets (images, videos, JS) at edge servers geographically close to users → reduces network round-trip time

Caching → stores pre-computed responses in fast memory (Redis / Memcached) → avoids slow DB queries

(d) Throughput:-

The number of requests a system can handle per unit time.

improvement

• Horizontal scaling

• Efficient algorithms

3 Load Estimation + load Balancing

Given

Daily Active users (DAU) = 500 million

API Requests per user per day = 50

Calculations

Total requests / day:

$$500,000,000 \times 50 = 25,000,000,000 \text{ (25 billion requests/day)}$$

Request / second:

$$25,000,000,000 / 86400 \text{ sec} = 289,352 \text{ RPS} \approx 290,000 \text{ RPS}$$

(Peak load = 2-3x average = $\approx 580,000 - 870,000 \text{ RPS}$)

Storage Estimation (assuming each request stores 1KB of data):

$$25 \text{ billion} \times 1 \text{ KB} = 25 \text{ TB/day}$$

$$\text{Per year: } 25 \text{ TB} \times 365$$

$$= 9125 \text{ TB}$$

$$\approx 9.1 \text{ PB/Year}$$

Load Balancing algorithms

Round robin: Requests distributed sequentially

Simple but ignores server load

Request 1 $\rightarrow$ Server A

R2 $\rightarrow$ SB

R3 $\rightarrow$ SC

R4 $\rightarrow$ SA

2) Random: Each request forwarded to a randomly selected server.
Request $\rightarrow$ [Random] $\rightarrow$ server B or A or C
simple no state needed 1x uneven distribution possible

3) Least connections: Request goes to the server with the fewest active connections.

SA: 10 active connections

SB: 3 "

" $\leftarrow$ new request goes here

Q4 Rest API & Hotel Booking system

sol Rest (Representational state transfer) is an architectural style for designing networked APIs using HTTP. A rest API exposes resources as URLs and uses standard HTTP methods.

6 Rest Architectural constraints

1. stateless — Each request contains all information needed
server stores no client session state

2. client-server — UI (client) and data storage (server) are separated: both can evolve independently

3. cacheable — Responses must define themselves as non-cacheable to improve perf.

4. uniform interface — consistent resource-based URLs, HTTP methods, and standard formats (JSON/XML)

5. layered system — client doesn't know if it's talking to the actual server or intermediary

6 Code on Demand server can send executable code (optional) (eg. Javascript) to clients

Hotel Booking System - API Design

Versioning Strategy: URL Versioning - /api/v1/

(i) Get Hotel Details

Get /api/v1/hotels/{hotel-id}

Response 200 OK:

```
{
  "hotelId": "H123",
  "name": "Grand Palace",
  "rooms": [
    {
      "roomType": "Delux",
      "available": true,
      "price": 3500
    }
  ]
}
```

(ii) Create Booking →

Request Body:

```
{
  "hotel_id": "H123",
  "room-type": "Delux",
  "check-in": "2026-05-01",
  "check-out": "2026-05-05",
  "user-id": "U456"
}
```

HTTP status code used:

200 OK - successful Get

201 Created - successful Post

400 Bad Request: invalid input

401 Unauthorized - Not authenticated

404 Not found - Resource doesn't exist

500 Internal server error - server failure.

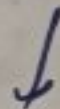
§5 Caching strategies

Caching is the process of storing frequently accessed data in a fast-access storage layer (cache) to reduce latency and database load

a) Cache - Aside (Lazy loading)

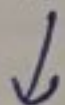
Application manages the cache manually

Read Request



[Cache] - Hit → Return Else

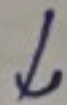
Miss



Database → fetch data



cache ← store data



Return data to user

(b) Read-through cache

cache sits between application and database; cache fetches missing data automatically.

Application $\rightarrow$ Cache $\rightarrow$ (on miss) $\rightarrow$ Database $\rightarrow$ cache $\rightarrow$ Application

c) Write-Through cache $\rightarrow$

Data is written to cache AND database simultaneously

Write Request

$\rightarrow$ [Cache] (write)

$\rightarrow$ [Database] (write)

$\rightarrow$ cache always consistent with DB

d) Write-Behind (write-back) cache $\rightarrow$

Data written to cache first; Database updated asynchronously after a delay.

Write Request $\rightarrow$ [Cache] - (async, later) $\rightarrow$ [Database]
very fast writes

Python LRU cache implementation

from collections import OrderedDict

class LRUcache:

def __init__(self, capacity: int):

self.capacity = capacity

self.cache = OrderedDict()

def get(self, key: int) $\rightarrow$ int:

if key not in self.cache:

return -1

self.cache.move_to_end(key)

return self.cache[key]

def put(self, key: int, value: int) $\rightarrow$ None:

if key in self.cache

self.cache.move_to_end(key)

self.cache[key] = value

if len(self.cache) > self.capacity:


```

self.cache.popitem(last=False)
ru = LRU(cache=13)
ru.put(1, 10)
ru.put(2, 20)
ru.put(3, 30)
print(ru.get(1))
ru.put(4, 40)
print(ru.get(2))

```

Q.6 Monolithic vs Microservices Architecture.

comparison

Parameter

Monolithic

Microservices

Deployment

single deployable unit: one change = full redeploy

Each services deploys independently

Scalability

scale entire app even if one module is the bottleneck

scale individual services as method

Fault Isolation

One bug can crash the entire application

failure in one services doesn't affect others

Team Ownership

All teams work on the same codebase - coordination overhead

Each service team owns a separate services

Technology

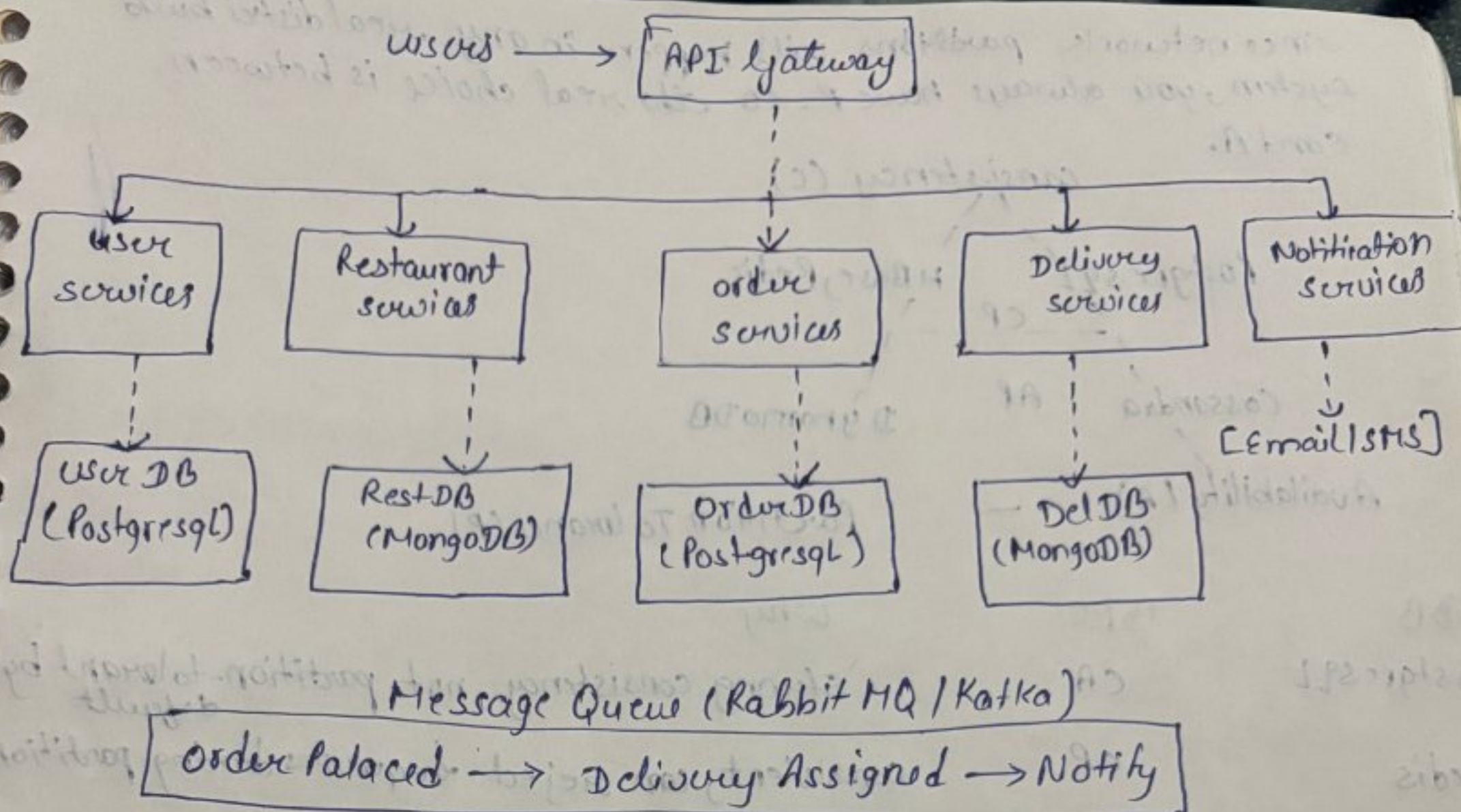
single language/framework

Each services can use different tech stack

Complexity

single simple to develop initially

complex (service discovery, distributed tracing)



Component Roles:

API Gateway: single entry point; handles auth, routing, rate limiting

user services: Registration, login, profiles

Restaurant Services: Menu management, restaurant info

Order Service: order placement, status tracking

Delivery services: Driver assignment, location tracking

Notifications services: sends SMS/email/push notifications

Message Queue (Kafka): Async communication between services

Q7: CAP Theorem

→ CAP Theorem states that in a distributed system, it is impossible to simultaneously guarantee all three properties:

Property

Definition

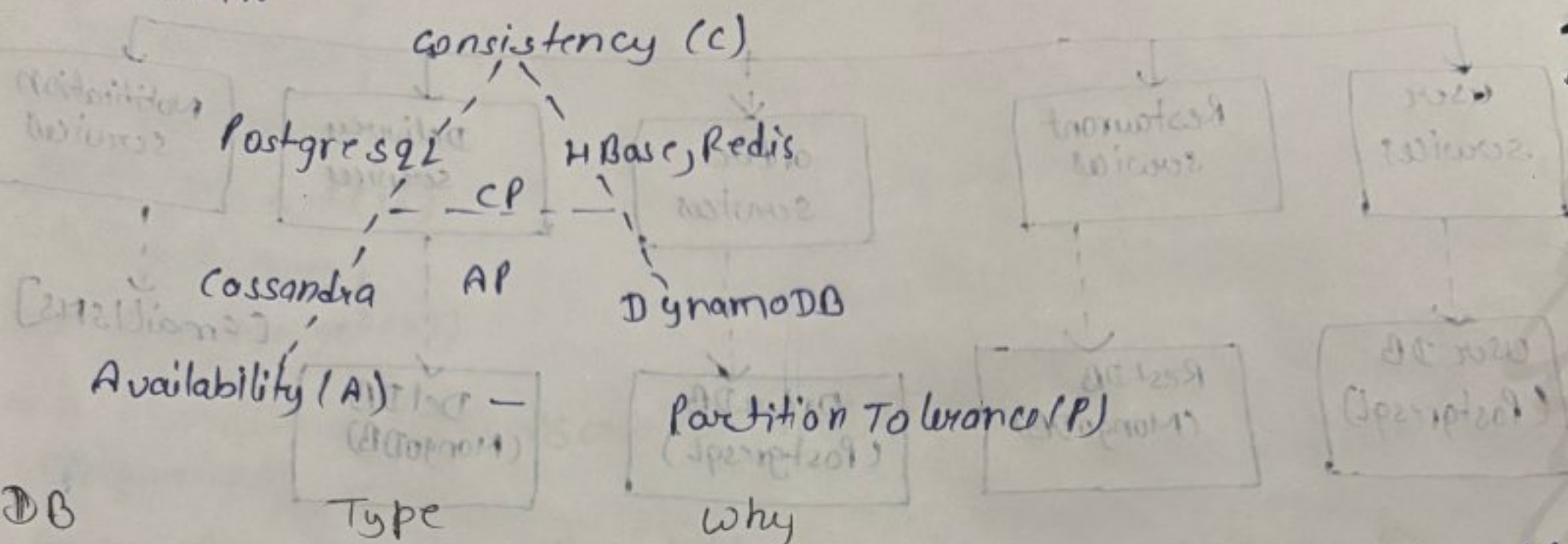
C- Consistency

Every read receives the most recent write or an error

A- Availability

Every request receives a (non-error) response, even if it may be stale

Since network partitions will happen in any real distributed system, you always have P. So the real choice is between C and A.



DB	Type	Why
PostgreSQL	CA	Strong consistency, not partition-tolerant by default
Redis	CP	Consistent, can reject requests during partition
HBase	CP	Strong consistency, maybe unavailable during failure
MongoDB	CP (default)	Can tune to AP
Cassandra	AP	Always available, eventually consistent
DynamoDB	AP (default)	Eventually consistent, strong consistency optional

SQL vs NoSQL + Database selection

Parameter	SQL	NoSQL
Data model	Table (row & columns)	Documents (key-value, graph column-family)
Schema	Fixed, defined upfront	Flexible / dynamic
Scalability	Vertical	Horizontal
Transactions	ACID	BASE (eventually consistent)
Query language	SQL (standardized)	Varies by DB

Joins	supported	Not supported
Consistency	strong	Eventual
Best for	financial, structured data	Big data, real-time apps

for 1 Billion user social media platform:

- User profiles $\rightarrow$ PostgreSQL structured data (name, email, bio), needs ACID, complex queries - relational DB is perfect.
- Posts/feeds $\rightarrow$ Cassandra Billion of posts, time-ordered, write-heavy - Cassandra's wide-column model handles this brilliantly at scale.
- Real-time chat $\rightarrow$ Redis for active low-latency messaging (Pub/sub). Cassandra for strong storing message history at scale.

09. Replication, sharding & consistent Hashing

Master-slave Replication: one master handles all writes. Data is copied to slave nodes which handle reads.

[Master] $\leftarrow$ writes
 $\hookrightarrow$ replications

[slave 1] [slave 2] [slave 3] $\leftarrow$ reads

Problem: master is a single point of failure

Master-Master Replication: Both nodes accept reads and write and sync with each other

[Master 1] $\rightarrow$ [Master 2]

Problem: conflict resolution when both writes same data simultaneously.

Range Based sharding: split data by range of values

IDs: 1-10M $\rightarrow$ shard 1

IDs: 10M-20M $\rightarrow$ shard 2

Problems: uneven load if one range gets more traffic

Hash-Based sharding: $\text{shard} = \text{hash}(\text{key}) \% N$ Distribute data

evenly: Problems: adding a node requires rehashing everything

Consistent Hashing: solves the rehashing problem. Both nodes

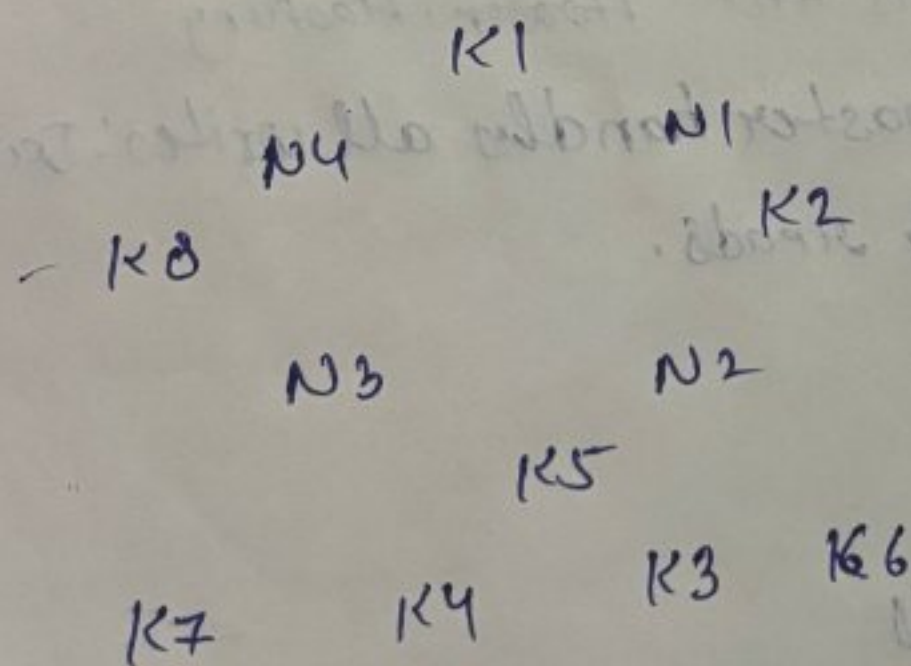
and keys are placed on a circular ring. A key goes to the

first node clockwise from it. when a node is added or

removed, only the key near that node are affected - not

everything.

Ring with 4 nodes, 8 keys:



Each key maps to the nearest clockwise node. Adding a node only moves a few nearby keys.

10 URL shortening service (like bit.ly)

Functional Requirements:

User gives a long URL $\rightarrow$ System returns a short URL

short URL redirects to original

Optional: track click count

Non functional:

Handle 100M URI s/day

low latency (<10ms for redirects)

99.99% availability

Load Estimation:

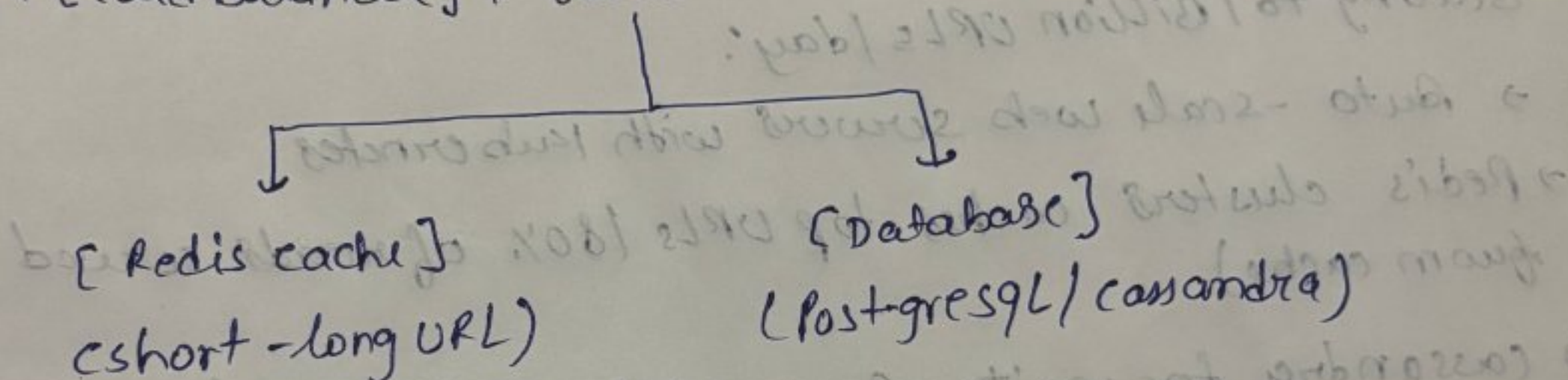
write: $100M / 86400 = 1,157 \text{ writes/sec}$

Assuming 100:1 read/write ratio: reads: $\approx 115,700 \text{ reads/sec}$

Storage (500 bytes per URL): $100M \times 365 \times 500B \approx 1.8TB/\text{year}$

Architecture $\rightarrow$

User $\rightarrow$ [Load Balancer] $\rightarrow$ [web servers]



Database schema:

column

short-code

long-url

user-id

created-at

click-count

Type
Var char (7) PK

Text

UUID

Timestamp

Bigint

Base 62 Encoding

uses characters: 0-9, a-z, A-Z = 62 chars

7 characters $\rightarrow 62^7 \approx 3.5 \text{ trillion unique codes}$

Base 62 = "0123456789abcdefghijklmnopqrstuvwxyz" ———

```
def encode(num, length=7):
```

```
    result = []
```

```
    while num > 0:
```

```
        result.append(BASE62[num%62])
```

```
        num //= 62
```

```
    return ''.join(reversed(result)).zfill(length)
```

Take MD5 hash of the long URL → convert to number

→ Base 62 encoder → get 7-char short code.

Scaling to 1 Billion URLs/day:

→ Auto-scale web servers with Kubernetes

→ Redis clusters (caches top URLs 180% of reads served from cache)

→ Cassandra for write-heavy DB at this scale

→ Snowflake ID generator for collision-free distributed id generation.